

Exercícios Aula 04

1. O Prontuário Digital (Dicionários)

Contexto: No Instagram, cada perfil tem nome e seguidores. No hospital, cada paciente tem ficha técnica.

Tarefa: Crie um dicionário chamado "paciente_atual" que armazene: nome, idade e glicemia. Exiba na tela apenas a glicemia deste paciente com uma mensagem personalizada.

2. Ronda na Estufa (Laço for)

O Problema (Problematização): Imagine que você é responsável pelo transporte de um lote de vacinas valiosas. Se a temperatura sair da faixa de segurança, o imunizante perde a eficácia e o trabalho de meses é perdido. Para garantir que tudo chegou bem ao destino, um sensor dentro da maleta térmica registra a temperatura a cada quilômetro.

Sua Tarefa (Ação): Temos a lista `rastreio_vacina = [2.2, 2.1, 5.1, 2.7, 4.0, 3.2, 4.5, 5.1, 2.8, 4.8]`. Você precisa criar um código que percorra essa lista e gere um relatório de segurança para o motorista, exibindo cada registro de forma profissional.

3. Triagem de Hemoglobina (Loop + Condicional)

Contexto: Análise de risco de anemia.

Tarefa: Percorra a lista `hemoglobinas = [13.5, 10.2, 11.0, 15.1, 9.5]`. Se o valor for menor que 12.0, exiba: "Atenção: Possível quadro de anemia". Caso contrário, exiba: "Nível normal".

4. O Protocolo de Diluição (Funções)

Contexto: Transformando POPs (Procedimentos Operacionais Padrão) em código.

Tarefa: Crie uma função chamada `calcular_diluicao` que receba o `volume_amostra` e o `fator_diluicao`. A função deve retornar o volume final. Teste-a com 2ml e fator 10.

5. Analista Automático (Integração)

Contexto: Integrando ferramentas de decisão.

Tarefa: Dada a função de colesterol abaixo, use um laço for para processar a lista `resultados = [180, 220, 250, 190]` e exibir o veredito para cada item.

```
def checar_colesterol(valor):  
    if valor > 200: return "Elevado"  
    return "Normal"
```

6. Conserto de Código (Debugging)

Contexto: Identificação de erros de lógica e indentação.

Tarefa: Corrija os erros do código abaixo para que ele conte corretamente os exames.

```
exames = ["Sangue", "Urina", "Fezes"]
total = 0
for ex in exames:
    print("Processando...")
total = total + 1
print(total)
```

```
[1]: exames = ["Sangue", "Urina", "Fezes"]
    total = 0
    for ex in exames:
        print("Processando...")
        total = total + 1
        print(total)
```

Cell In[1], line 4
 print("Processando...")
 ^
IndentationError: expected an indented block after 'for' statement on line 3

7. O Cenário (Problematização)

Houve uma suspeita de contaminação por metais pesados no reservatório principal que abastece o Hospital Universitário. Como biomédico da vigilância sanitária, você recebeu uma planilha com 1.000 amostras coletadas em diferentes pontos da rede. Se você demorar para analisar, o hospital terá que ser evacuado. Fazer isso manualmente levaria dias; você tem apenas alguns minutos.

Sua Missão: Você deve criar um sistema automático que leia essa lista gigante e aplique o Protocolo de Segurança Ambiental.

Passo 1: O Protocolo (A Função)

Crie uma função chamada `analisar_amostra`. Ela deve receber um valor (nível de contaminação) e retornar:

- "SEGURO": se o valor for menor que 20.0.
- "ALERTA": se o valor estiver entre 20.0 e 50.0.
- "TOXICO": se o valor for maior que 50.0.

Passo 2: A Varredura (O Loop)

Use um laço `for` para percorrer a lista de 1.000 valores fornecida abaixo. Para cada amostra, a sua função deve ser chamada.

Passo 3: O Relatório Final (A Autonomia)

Ao final do loop, o programa deve exibir:

1. Quantas amostras foram classificadas como TÓXICO.
2. Uma mensagem final: "Evacuação Necessária" (se houver mais de 10 amostras tóxicas) ou "Situação sob Controle" (se houver 10 ou menos).

```
import random

# 0 Big Data da Missão: 1.000 amostras do reservatório
valores_amostras = [round(random.uniform(0.0, 100.0), 1) for _ in range(1000)]

# 1. Defina aqui sua função (O Protocolo POP)
def analisar_amostra(nivel):
    # Seu código aqui
    pass

# 2. Crie as variáveis para contar os resultados
contagem_toxico = 0

print("🔍 Iniciando análise em massa do reservatório...")

# 3. Desenvolva aqui o seu Laço for
# Chame a função dentro do loop e atualize a contagem_toxico se necessário

# 4. Exiba o veredito final baseado na contagem
```